Preprocession Experiment

It breaks down the preprocessing pipeline into 6 choices, which will be measured, each of these choices have caused a change with the corpuses index. For each choice, the code builds an in memory SPIMI index as it records its counts and moves on to the next. The measurements are the # of distinct terms and nonpositional postings.

All choices use the same 21,578 articles, NEWIDs, and title and body of text. The tokenizer uses ASCII letters as its reference of extraction and the postings represent a term to document pair, having no duplicate of terms.

Construction

There 6 choices/layers:

1. unfiltered : no removal of anything
2. No numbers: removing any tokens that hits true using the str.isdigit() function
3. Case folding: lowercase the tokens
4. 30 stop words: remove the most frequent token from filtered (filtered from the prev steps) corpus
5. 150 stop words: remove the most frequent token from the same filtered corpus (The stop words are constructed and ranked by total token occurrences)
6. Stemming: We apply NLTKs porter stemmer to the more filtered corpus (from 1-5)

Results and Reflection

Preprocessing	Unique terms	Change %	Total Change	Postings	Change %	Total Change
unfiltered	61,453	//	//	1,781,942	//	//
No numbers	59,252	-3.6%	-3.6%	1,622,359	-9.0%	-9.0%
Case folding	43,197	-27.1%	-29.7%	1,499,117	-7.6%	-15.9%
30 stop words	43,167	-0.1%	-29.8%	1,218,450	-18.7%	-31.6%
150 stop words	43,047	-0.3%	-30.0%	930,958	-37.9%	-47.8%
Stemming	32,318	-24.9%	-47.4%	886,223	-4.8%	-50.3%

What this shows:

- removing numbers has little effect on distinct terms.
- Case folding combining and stemming both tremendously reduce the vocabulary.
- Stop word removals both 30 and 150 behave differently, in 150 it reduces by 37.9% while with distinct terms its only 0.3%.

Comparisons (Table 5.1)

The general trend of our finding follow the patterns of table 5.1. Where the general trends of what affects the index and what does not. But in terms of the magnitudes, thats where they differ. Case folding (in table 5.1) reduces terms and postings by 17% and 3% respectively. While with our findings we see 27.1% and 7.8% respectively. Although they differ this is to be expected, since the corpus of table 5.1 is different from ours, and since the corpus is different the tokenizer and preprocessing choices differ in the finer details, accumulating to the differences of magnitudes. This shows the usefulness of table 5.1 in its trends but not to be expected in it matching exactly.

Implementation:

tokenizer.py: The tokenizer
compress.py: Stop word ranking, variant definitions and index count are all handled here

To run the project tests use this command:

- python -m unittest test_tiny.py

And to print the posting and term counts from the repos root use:

- python demo.py compression